

Model Package

The `model` package defines the Data Transfer Objects (DTOs) and domain entities used throughout the application. These classes are designed to be serialized/deserialized into JSON, following the TMF621 (Trouble Ticket API) specification.

Core Models

`TroubleTicket.java`

The main entity representing a Trouble Ticket.

```
public class TroubleTicket {
    private String id;
    private String status;
    private List<Note> note = new ArrayList<>();
    // ... other fields

    @JsonProperty("@type")
    private String type;

    public TroubleTicket() {
        this.id = UUID.randomUUID().toString();
        this.status = "Acknowledged";
        // ... initialization
    }
}
```

Explanation

- **Fields:** Contains all attributes of a ticket such as `id`, `name`, `status`, `severity`, and complex sub-objects like `List<Note>`, `List<Attachment>`, etc.
 - **Constructor:** Automatically generates a unique `id` using `UUID` and sets default values like `status = "Acknowledged"`.
-

Supporting Models

The following classes represent sub-components of a `TroubleTicket`:

- **`Attachment.java`:** Represents external files or links associated with a ticket.
- **`Note.java`:** Represents comments or annotations added to a ticket. It automatically sets an `id` and a `date` upon creation.

- **RelatedParty.java & RelatedEntity.java:** Link the ticket to customers, users, or other systems.
 - **StatusChange.java:** Tracks the history of status transitions. Uses an `idCounter` for simple ID generation.
 - **TicketRelationship.java:** Defines relationships between different tickets (e.g., dependency, sub-task).
 - **Channel.java:** Indicates the source of the ticket (e.g., web, mobile app).
 - **ErrorResponse.java:** Used by the exception package to format error messages consistently.
-

Analysis

Annotations

- **@JsonProperty:** Used to map Java field names to specific JSON keys. This is particularly important for fields starting with `@` (e.g., `@type`, `@baseType`, `@referredType`), which are reserved characters in Java identifiers but required by the TMF specification.

Data Types

- **List<T> (ArrayList):** Favored for collections like `note`, `attachment`, and `relatedParty`. `ArrayList` is chosen for its performance in frequent read operations and its ability to maintain the order of elements (e.g., notes in chronological order).
- **String for IDs:** `UUID.randomUUID().toString()` is used for `id` fields to ensure globally unique identifiers that are easy to transmit over HTTP.
- **Double for size:** Used in `Attachment` to represent file sizes, allowing for fractional values.

Architectural Decisions

- **Default Values in Constructors:** By initializing fields like `id`, `creationDate`, and `status` in the constructor, the application ensures that every new `TroubleTicket` object is in a valid starting state before it even reaches the service layer.
- **Separation of Concerns:** Each class represents a single logical entity from the domain model, keeping the code modular and easy to maintain.